Reference Articles for the Inventory Server Project

Group 4 — Inventory Management System (Java SE 8 / com.sun.net.httpserver)

This document lists external references that align with the coding patterns, architecture, and design decisions adopted in our project. Each entry describes what the resource covers and which aspect of our codebase directly adopts that pattern.

1. com.sun.net.httpserver — Lightweight HTTP Server API

Oracle: com.sun.net.httpserver Package Summary

https://docs.oracle.com/javase/8/docs/jre/api/net/httpserver/spec/com/sun/net/httpserver/package-summary.html

What it covers: Official Javadoc for HttpServer, HttpHandler, HttpExchange, HttpContext, and Filter classes. Includes working examples of creating an embedded HTTP server.

Adopted in our project: Our entire server infrastructure — HttpServerBootstrap, Router (implements HttpHandler), RequestContext (wraps HttpExchange), and the filter chain — is built directly on this API.

Oracle: Filter Class Documentation

https://docs.oracle.com/javase/8/docs/jre/api/net/httpserver/spec/com/sun/net/httpserver/Filter.html

What it covers: The abstract Filter class, Filter.Chain, and how to pre/post-process exchanges.

Adopted in our project: AuthFilter, CorsFilter, and LoggingFilter all extend this class, using doFilter(HttpExchange, Chain) to form the middleware pipeline.

Baeldung: Simple Web Server in Java

https://www.baeldung.com/simple-web-server-java-18

What it covers: HttpServer, HttpHandler, and Filter concepts. Although it references Java 18's SimpleFileServer, the underlying classes (HttpServer.create, createContext, Filters) are the same API we use.

Adopted in our project: The pattern of HttpServer.create() → createContext() → attach filters → set executor → start() is exactly how HttpServerBootstrap.start() works.

2. Layered Architecture (Handler → Service → Repository)

Martin Fowler: Repository Pattern (PoEAA)

https://martinfowler.com/eaaCatalog/repository.html

What it covers: Mediates between domain and data mapping layers using a collection-like interface.

Adopted in our project: UserRepository, SessionRepository, InquiryRepository all encapsulate raw JDBC calls behind clean method signatures (findByUsername, create, invalidate).

Baeldung: Controller, Service, and DAO Example

https://www.baeldung.com/jsf-spring-boot-controller-service-dao

What it covers: The 3-tier pattern where Controllers handle HTTP, Services hold business logic, and DAOs/Repositories manage persistence.

Adopted in our project: Our Handlers (e.g. LoginHandler) delegate to AuthService, which delegates to UserRepository/SessionRepository. Each layer has a single responsibility.

3. DTO Pattern (Data Transfer Objects)

Baeldung: The DTO Pattern

https://www.baeldung.com/java-dto-pattern

What it covers: DTOs as flat POJOs carrying data between processes/layers, decoupling domain models from presentation, and reducing method call count.

Adopted in our project: All request/response objects live in dto/ packages — LoginRequest, LoginData, UserProfile, ApiResponse, ErrorResponse, WelcomeData, etc. They contain no business logic, only data and accessors.

4. Custom Exception Hierarchy for HTTP Errors

Baeldung: Custom Exceptions in Java

https://www.baeldung.com/java-new-custom-exception

What it covers: Creating exception class hierarchies, checked vs unchecked, adding context fields.

Adopted in our project: ApiException (base, carries HTTP status code) → UnauthorizedException (401), ForbiddenException (403), NotFoundException (404), ValidationException (422), ConflictException (409).

Reflectoring: Exception Handling in REST APIs

https://reflectoring.io/spring-boot-exception-handling/

What it covers: Mapping exception types to HTTP status codes centrally, structured error payloads, field-level validation errors.

Adopted in our project: Router.handle() catches ApiException/ValidationException and maps them to structured JSON ErrorResponse objects with status codes — same design rationale without Spring.

5. Filter/Middleware Chain (Chain of Responsibility)

Baeldung: Chain of Responsibility Pattern

https://www.baeldung.com/chain-of-responsibility-pattern

What it covers: Design pattern where request passes through a chain of handlers; each decides to process or delegate. Servlet Filters are cited as a real-world example.

Adopted in our project: LoggingFilter → CorsFilter → AuthFilter form a chain. Each calls chain.doFilter() to continue, or terminates early (e.g. AuthFilter returns 401 without proceeding).

Baeldung: What Is chain.doFilter() in Filters?

https://www.baeldung.com/java-spring-chain-dofilter

What it covers: How doFilter() delegates to the next filter; the difference between the filter's own doFilter and Chain.doFilter; connection to Chain of Responsibility.

Adopted in our project: Identical mechanism — our filters extend com.sun.net.httpserver.Filter and call chain.doFilter(exchange) to pass control forward.

6. JWT Authentication with JJWT

Baeldung: Supercharge Java Auth with JWTs (JJWT)

https://www.baeldung.com/java-json-web-tokens-jjwt

What it covers: Building JWTs with Jwts.builder(), parsing with Jwts.parserBuilder(), signing with HMAC-SHA256, adding custom claims, and expiration handling.

Adopted in our project: JwtUtil.generateToken() uses Jwts.builder().setClaims().setSubject().signWith(key).compact(). JwtUtil.validateToken() uses Jwts.parserBuilder().setSigningKey(key).build().parseClaimsJws(token).getBody().

7. Password Hashing with BCrypt

Baeldung: Hashing a Password in Java

https://www.baeldung.com/java-password-hashing

What it covers: Why MD5/SHA are insufficient, BCrypt/SCrypt/PBKDF2 as recommended algorithms, salt generation, and cost factor tuning.

Adopted in our project: PasswordUtil uses jBCrypt (org.mindrot.jbcrypt.BCrypt) with configurable cost from EnvConfig.passwordBcryptCost(). Default admin password is pre-hashed with cost 10.

8. Session Management & Authentication Security (OWASP)

OWASP: Session Management Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

What it covers: Server-side session storage, token entropy, session expiration, idle timeouts, and session invalidation on logout.

Adopted in our project: We store SHA-256 hash of JWT (never raw token) in user_sessions table. Sessions have expires_at, last_activity_at tracking, and are explicitly invalidated on logout.

OWASP: Authentication Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

What it covers: Credential storage, account lockout, generic error messages, password complexity, and brute-force protection.

Adopted in our project: AuthService uses generic 'Invalid username or password' messages, implements account lockout after N failed attempts (configurable), and stores BCrypt hashes.

OWASP: REST Security Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

What it covers: HTTPS, access control per endpoint, JWT best practices, input validation, CORS, HTTP status code usage, and error handling.

Adopted in our project: AuthFilter enforces token validation on every non-public route. CorsFilter handles cross-origin. ValidationUtil validates all input. Router maps exceptions to proper HTTP status codes (401, 403, 404, 422, 500).

9. HikariCP Connection Pooling

Baeldung: Introduction to HikariCP

https://www.baeldung.com/hikaricp

What it covers: HikariCP setup via HikariConfig, DataSource creation, pool sizing, connection timeout, and performance characteristics.

Adopted in our project: DatabaseConfig initializes HikariDataSource with all pool parameters (min/max pool size, connection timeout, idle timeout, max lifetime) read from EnvConfig.

HikariCP GitHub Wiki

https://github.com/brettwooldridge/HikariCP/wiki

What it covers: Pool sizing guidelines, configuration properties, and integration patterns.

Adopted in our project: Pool defaults (min=2, max=10) and configuration properties like cachePrepStmts are based on HikariCP's recommended settings.

10. Environment-Variable-Driven Configuration (12-Factor App)

The Twelve-Factor App: III. Config

https://12factor.net/config

What it covers: Strict separation of config from code. Config stored in environment variables, never in files checked into version control. Env vars are language/OS-agnostic and orthogonal.

Adopted in our project: EnvConfig.java reads 116 configuration values from G4IMS_SERVER_* environment variables with sensible defaults. No config files are committed. Each variable is independently managed per environment.

11. Google Java Style Guide

Google: Java Style Guide

https://google.github.io/styleguide/javaguide.html

What it covers: File structure, imports (no wildcards, ASCII-sorted), formatting (2-space indent, K&R; braces, 100-char column limit), naming (UpperCamelCase classes, lowerCamelCase methods), and Javadoc conventions.

Adopted in our project: Enforced automatically by fmt-maven-plugin (google-java-format) in the Maven build lifecycle. All 50 source files are formatted on every compile.

12. Maven Shade Plugin (Fat JAR Packaging)

Baeldung: How to Create an Executable JAR with Maven

https://www.baeldung.com/executable-jar-with-maven

What it covers: Section 2.3 covers maven-shade-plugin — creating uber-JARs that include all dependencies, configuring the main class via ManifestResourceTransformer, and shading.

Adopted in our project: pom.xml uses maven-shade-plugin to produce a single inventory-server-1.0.0.jar containing all dependencies. Deployed as java -jar without classpath setup.

13. Database Migrations (Flyway-Inspired Versioned Files)

Flyway GitHub Repository

https://github.com/flyway/flyway

What it covers: Versioned migration concept — files named V###__description.sql applied in order, tracked in a schema history table, never re-applied.

Adopted in our project: Custom MigrationRunner implements the same convention: V001__create_schema_migrations_table.sql through V004__seed_roles_permissions_admin.sql. Applied versions tracked in schema_migrations table. CLI commands: migrate, migrate:status, migrate:rollback, migrate:generate.